



CS1004 – Object Oriented Programming

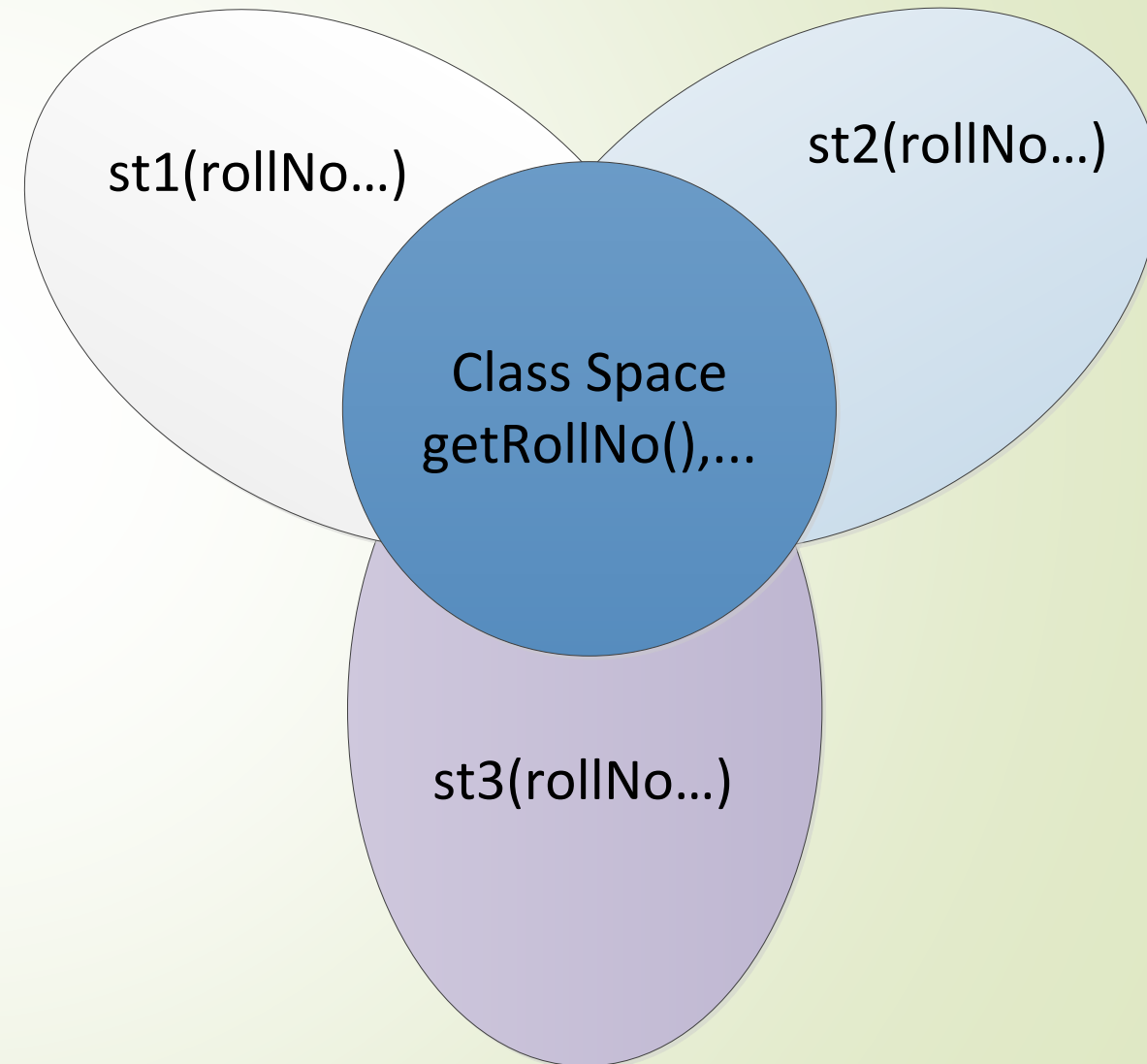
Lecture # 12
Tuesday, March 22, 2022
Spring 2022
FAST – NUCES, Faisalabad Campus

Muhammad Yousaf

2

Constant members

➤ `Student st1, st2, st3;`



Initializer list pitfalls

```
class my_string
{
private:
    char* str;
    unsigned int length;
public:
    my_string(const char* cp) {
        length = strlen(cp);
        str = new char[length + 1];
    }
};
```

- ➡ Member length initialized after str
- ➡ But str uses length...

```
my_string:: my_string(const char *t):
    length(strlen(t)),str(new char[length]+1)
{
    strcpy(str, t);
}
```

Copy constructor

➡ Initializes a new object from another, existing one

➡ Signature:

```
Class::Class(Class &obj)
{
}
```

Copy constructor for class Student

- Implicitly called in 3 situations
 - Defining a new object from an existing object
 - Passing an object by value
 - Returning an object by value

Copy constructor

```
int main()
{
    Student st1(15, 1024, "Rizwan");

    //Initialize 2 local objects from st1
    Student st2(st1); //pass by value
    Student st3 = st1; //return value

    //Initialize a dynamic object from st1
    Student *st4 = new Student(st1);

    return 0;
}
```

Copy constructor

```
bool compareTwo(Student, Student);
```

```
Student st1(15, 1024, "Rizwan");
```

```
Student st3(15, 1024, "Rizwan");
```

```
//Copy constructor will be called for both parameters
```

```
if (st1.compareTwo(st1, st3))
```

```
    cout << "Both are same\n";
```

Copy constructor

```
Student testFunc()  
{  
    Student st(25, 1654, "TestName");  
    return st; //Return by value will invoke copy constructor  
}
```


Copy constructor

```
// Copy constructor
```

```
Student::Student(const Student & myStudent)
```

```
{
```

```
    this->age = myStudent.age;
```

```
    this->rollNo = myStudent.rollNo;
```

```
    this->name = new char[strlen(myStudent.name) + 1];
```

```
    strcpy(this->name, myStudent.name);
```

```
}
```

Destructor

```
Student::~~Student()  
{  
    cout << "Destructor called\n";  
    if (name)  
        delete[] name;  
    name = nullptr;  
    noOfStudents--;  
}
```

Pointer to object

- ➡ A variable that holds an address value is called a pointer variable or simply pointer
- ➡ Pointer to objects are similar as pointer to built-in types

```
int main() {  
    Student obj;  
    Student *ptr;  
    ptr = &obj;  
    obj.setRollNo(5);  
    ptr->setRollNo(10);  
    (*ptr).setRollNo(10);  
    return 0;  
}
```

```
class Student {  
    ...  
public:  
    Student();  
    Student(char * aName);  
    void setRollNo(int aRoll);  
};
```

Allocation with new operator

- Sometimes we do not know, at the time that we write the program , how many objects we want to create.
- When this is the case we can use new to create objects while the program is running
- 'new' returns a pointer to unnamed objects

Example

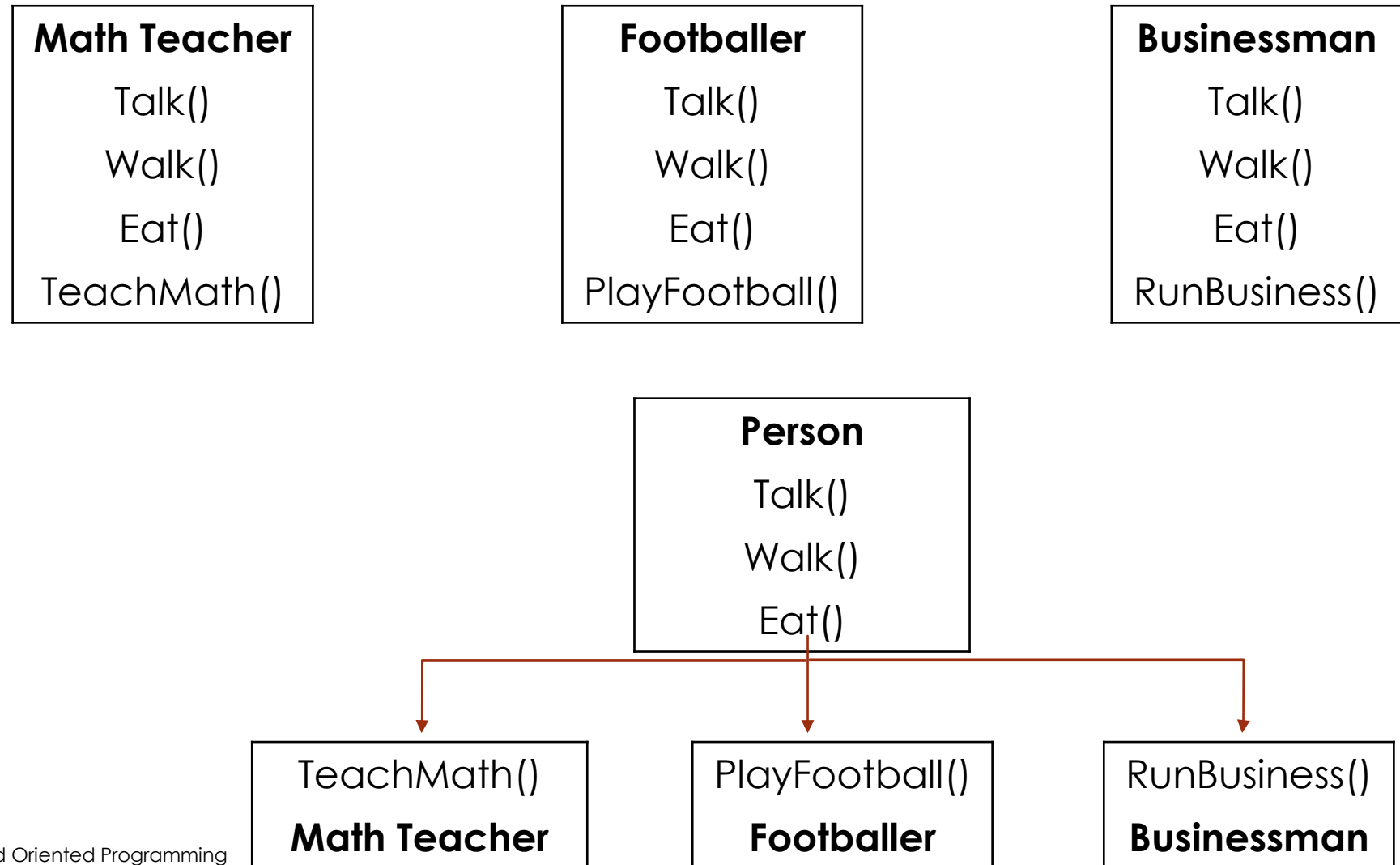
```
int main() {  
    Student *sPtr;  
    sPtr = new Student;  
    sPtr->setRollNo(10);  
    return 0;  
}
```



15

Inheritance

Inheritance



Introduction

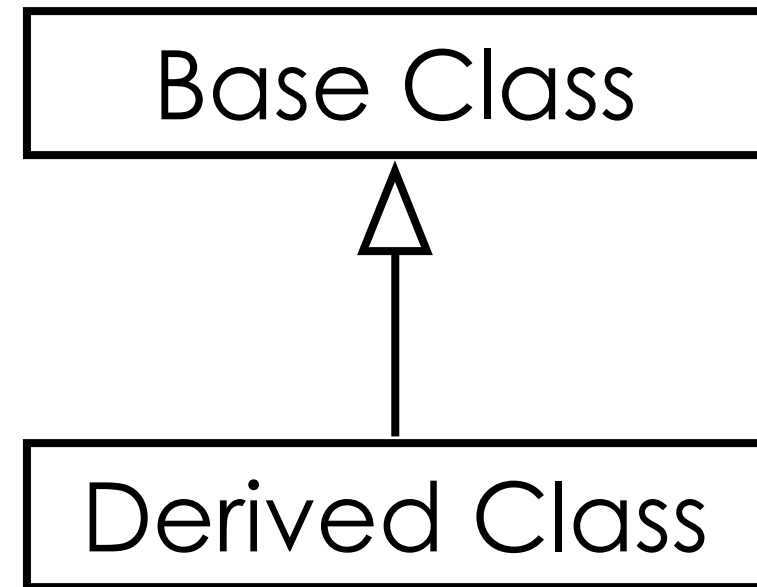
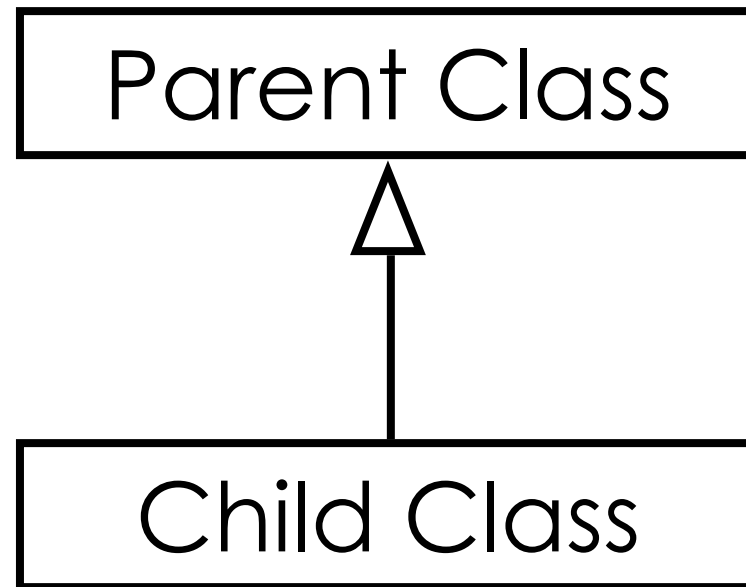
- Probably most powerful feature of OOP
- New classes are created from existing classes

Advantages

- Reusability
- Saves time and effort
- Improves program structure and reliability

Inheritance in classes

- ➡ If a class B inherits from class A, then B contains all the characteristics (information structure and behavior) of class A
- ➡ The parent class is called **base class** and the child class is called **derived class**
- ➡ Besides inherited characteristics, derived class may have its own unique characteristics



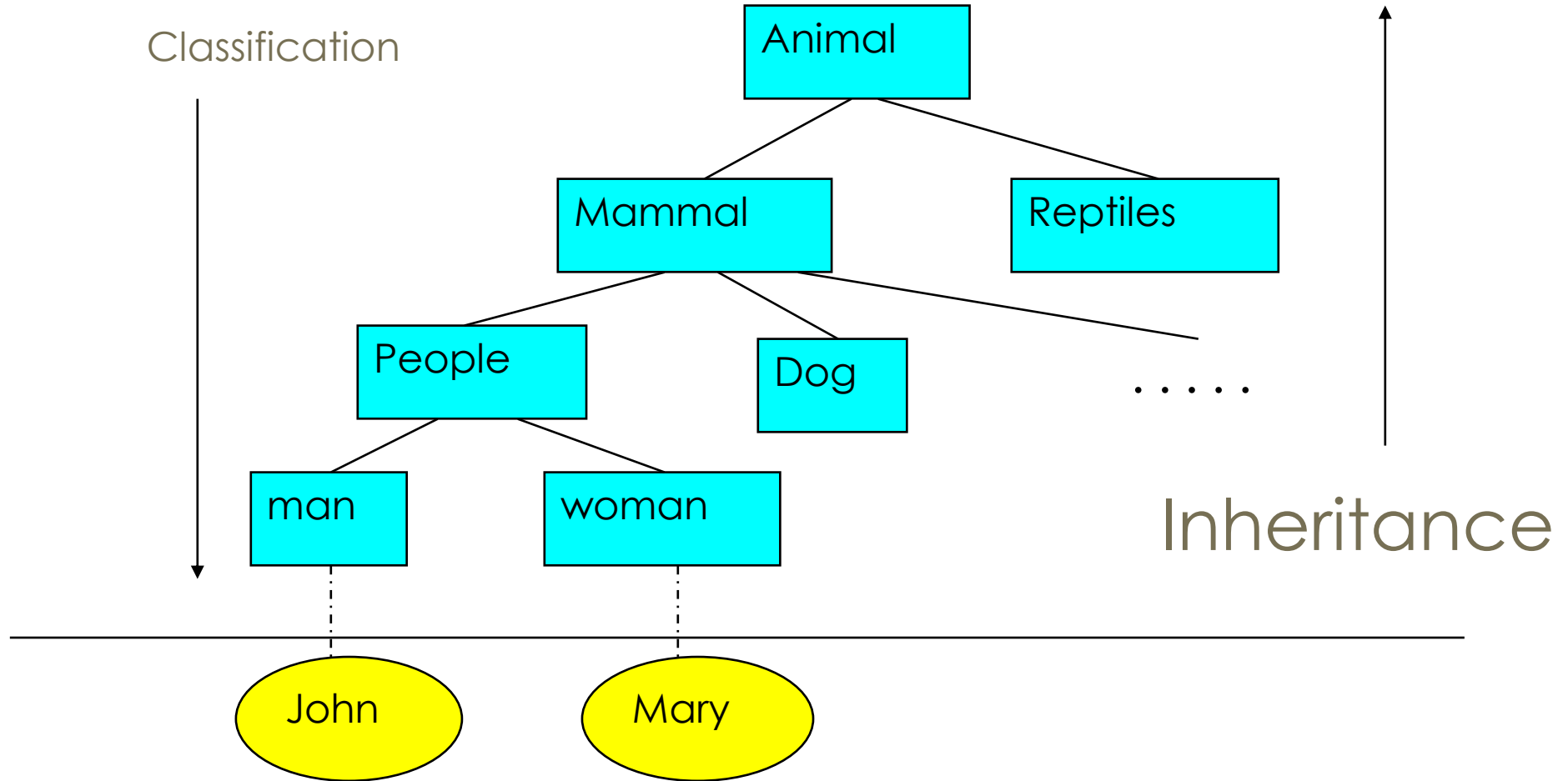
Inheriting Data and Functions

- ➡ All data members and member functions of base class are inherited to derived class
- ➡ Constructors, destructors and `= operator` are not inherited

Some definitions in class hierarchy

- Direct base class
 - Inherited explicitly (one level up hierarchy)
- Indirect base class
 - Inherited two or more levels up hierarchy
- Single inheritance
 - Inherits from one base class
- Multiple inheritance
 - Inheritance from multiple classes

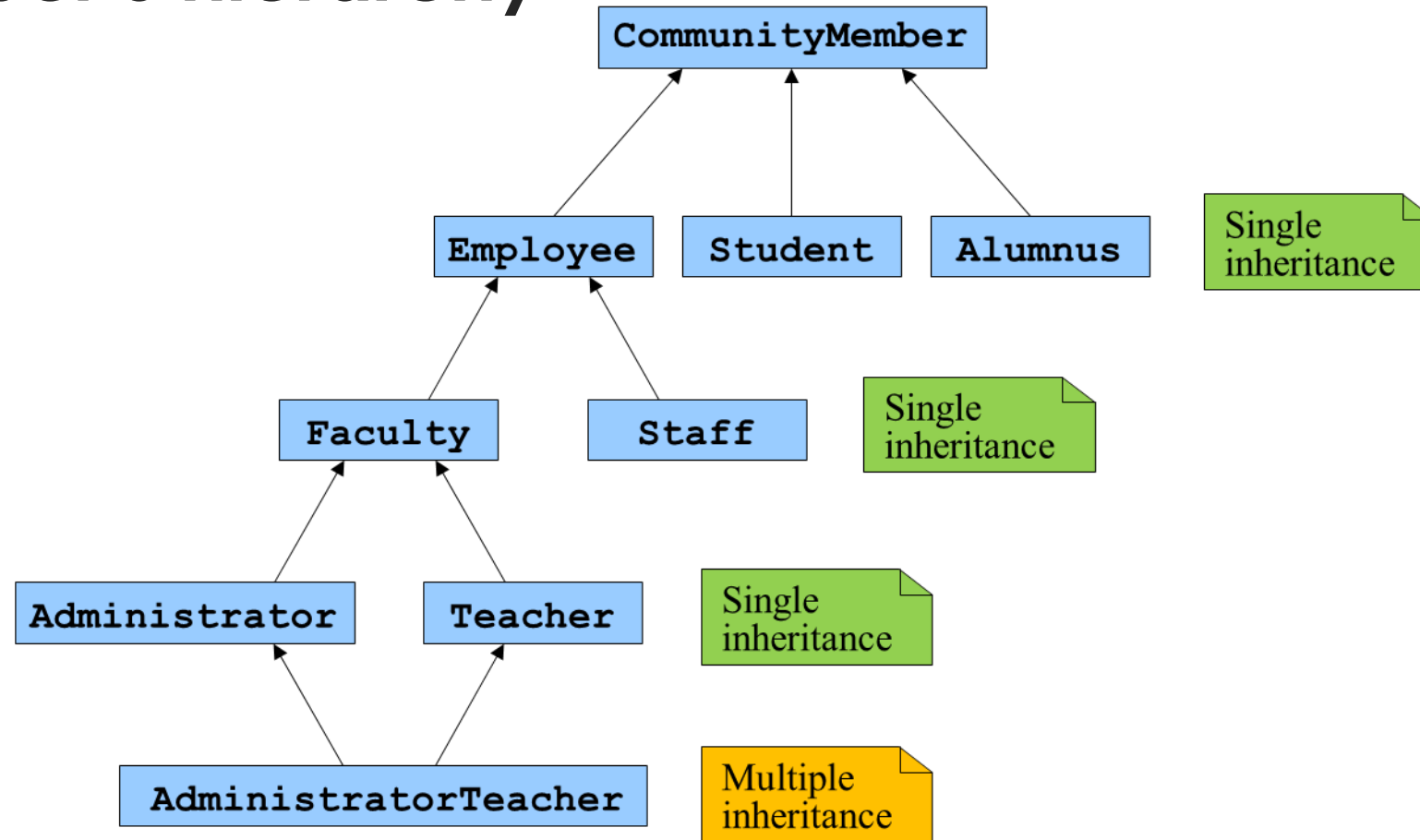
Animals: Class's hierarchy



Inheritance Examples

Base Class	Derived Class
Student	GraduateStudent UndergraduateStudent
Shape	Circle Triangle Rectangle
Loan	CarLoan HomeImprovementLoan
Employee	StaffMember FacultyMember
Account	CheckingAccount SavingAccount

Another example: University's community member's hierarchy



Inheritance in C++

- ➡ There are three types of inheritance in C++
 - ➡ Public
 - ➡ Private
 - ➡ Protected

Public Inheritance

- ➡ With **public inheritance**,
 - ➡ **public** and **protected** members of the base class become respectively **public** and **protected** members of the derived class

Protected Inheritance

➡ With **Protected inheritance**

➡ **Public** and **Protected** members of the base class become **Protected** members of the derived class

Private Inheritance

- ➡ With **private inheritance**
 - ➡ **Public** and **protected** members of the base class become **private** members of the derived class

Three modes of inheritance

```
class base
{
public:
    int x;
protected:
    int y;
private:
    int z;
};

class publicDerived : public base
{
    // x is public
    // y is protected
    // z is not accessible from
    //publicDerived
};
```

```
class protectedDerived : protected base
{
    // x is protected
    // y is protected
    // z is not accessible from
    //protectedDerived
};

class privateDerived : private base
{
    // x is private
    // y is private
    // z is not accessible from
    //privateDerived
}
```

Base class member access qualifier	Type of inheritance		
	Public inheritance	Protected inheritance	Private inheritance ⁺
Public	Public in derived class. Can be accessed directly by any non-static member function, friend function and non-member function	Protected in derived class. Can be accessed directly by any non-static member function and friend function	Private in derived class. Can be accessed directly by any non-static member function and friend function
Protected	Protected in derived class. Can be accessed directly by all non-static member functions and friend functions	Protected in derived class. Can be accessed directly by all non-static member functions and friend functions	Protected in derived class. Can be accessed directly by all non-static member functions and friend functions
Private	Hidden in derived class. Can be accessed by non-static member functions and friend functions through public or protected member functions of base class	Hidden in derived class. Can be accessed by non-static member functions and friend functions through public or protected member functions of base class	Hidden in derived class. Can be accessed by non-static member functions and friend functions through public or protected member functions of base class

Define a Class Hierarchy

➤ Syntax:

```
class DerivedClassName : access-level BaseClassName
```

➤ Where

- access-level specifies the type of derivation
- private by default, protected or public

➤ Any class can serve as a base class

➤ Thus a derived class can also be a base class

Questions

